

pateda: A Python Library for Estimation of Distribution Algorithms

User Guide — Version 1.0 (Draft)

Roberto Santana
Department of Computer Science and Artificial Intelligence
University of the Basque Country (UPV/EHU)
`roberto.santana@ehu.eus`

2026

Abstract

This document describes **pateda**, a Python library for estimation of distribution algorithms (EDAs). **pateda** supports single- and multi-objective optimization over discrete and continuous search spaces. The library provides (i) a set of plug-and-play algorithm classes that encapsulate complete EDA pipelines, (ii) a flexible component-based architecture that allows the user to mix and match any combination of selection, learning, sampling, replacement, and other operators, and (iii) a knowledge-extraction module for analyzing probabilistic models learned during the evolutionary search. **pateda** is the Python successor to MATEDA-2.0 (Santana et al., 2009), inheriting its modular philosophy while taking advantage of modern Python scientific computing libraries.

Contents

1	Introduction	3
2	Installation and Package Structure	4
2.1	Installation	4
2.2	Package Structure	4
2.3	Running the Examples	4
3	Defining and Executing an EDA	5
3.1	General Description	5
3.2	The EDA Class	5
3.2.1	Input Parameters	6
3.2.2	Output: Statistics and Cache	6
3.3	The EDAComponents Dataclass	6
3.4	Example: UMDA on the OneMax Problem	7
3.5	Example: Gaussian UMDA on the Sphere Function	7
3.6	Fitness Convention	8
4	Plug-and-Play Algorithm Classes	8
4.1	Discrete Algorithm Classes	8
4.2	Continuous Algorithm Classes	8
4.3	Quick Comparison Example	9

5	Probabilistic Models and Algorithm Families	10
5.1	Discrete Representation	10
5.2	Continuous Representation	10
5.3	Factorized Distributions	10
5.4	Univariate Models	11
5.5	Tree and Forest Models	11
5.6	Bayesian Network Models	12
5.7	Markov Network Models	12
5.8	Markov Chain Models	13
5.9	Constrained EDAs	13
5.10	Continuous Gaussian Models	13
6	EDA Components	13
6.1	Seeding Methods	13
6.2	Selection Methods	14
6.3	Replacement Methods	14
6.4	Stopping Conditions	14
6.5	Local Optimization	14
6.6	Repairing Methods	14
7	Test Functions and Benchmark Problems	15
7.1	Discrete Functions	15
7.2	Continuous Functions	15
8	Multi-Objective Optimization	15
8.1	Multi-Objective Problem Definition	15
8.2	Pareto Dominance	16
8.3	Multi-Objective Selection	16
8.4	Multi-Objective EDA Pipeline	16
8.5	Extracting Pareto Fronts	16
9	Knowledge Extraction	17
9.1	Dependency Analysis	17
9.2	Fitness Measures	17
9.3	Network Measures and Visualizations	17
9.4	Continuous Variable Analysis	18
10	Injecting A-Priori Knowledge	18
10.1	Fixed Factorizations	18
10.2	Interaction Matrices	18
10.3	Biased Initialization	18
10.4	Constrained Search	19
10.5	Selection Weighting	19
11	Conclusions	19

1 Introduction

Estimation of distribution algorithms (EDAs) (Larrañaga and Lozano, 2002; Lozano et al., 2006; Mühlenbein and Paaß, 1996; Pelikan et al., 2006) are a class of evolutionary algorithms that replace the variation operators of traditional genetic algorithms (GAs) (Goldberg, 1989; Holland, 1975) with the explicit estimation and sampling of probabilistic models. At each generation, a probabilistic model is learned from a selected subset of the current population; new individuals are then generated by sampling from this model. The probabilistic model captures the statistical dependencies between the problem variables that tend to co-occur in high-quality solutions, endowing EDAs with the ability to exploit problem structure in a principled way.

Several classes of probabilistic models have been proposed for EDAs. They range from simple univariate (independent-variable) models (Mühlenbein and Paaß, 1996; Baluja, 1994) through tree-structured models (Baluja and Davies, 1997; Pelikan and Mühlenbein, 1999) and Bayesian networks (Etxeberria and Larrañaga, 1999; Pelikan et al., 1999; Pelikan, 2005) to Markov networks (Santana, 2003, 2005; Shakya and McCall, 2007; Shakya and Santana, 2008), factorized distributions (Mühlenbein et al., 1999; Harik, 1999), and mixtures of distributions (Bosman and Thierens, 2002). For continuous optimization, Gaussian EDAs (Larrañaga et al., 1999, 2000; Bosman and Thierens, 2000b,a) and mixture-of-Gaussian models are the most common choices.

Beyond optimization, the probabilistic models learned by EDAs encode information about the problem structure that can be extracted and analyzed. This *knowledge extraction* perspective has been pursued in a number of works (Echegoyen et al., 2007, 2008; Hauschild and Pelikan, 2008; Lima et al., 2007; Santana et al., 2007) and is an important feature of *pateda*.

pateda is a Python 3 library that implements a wide range of EDAs for discrete and continuous single- and multi-objective optimization. It is the Python successor to MATEDA-2.0 (Santana et al., 2009), a Matlab suite with a similar modular philosophy. The main design goals of *pateda* are:

- **Plug-and-play convenience.** Algorithm classes such as `UMDA`, `EBNA`, `GaussianUMDA`, and `GaussianNetworkEDA` provide a one-liner interface for the most common EDA configurations.
- **Flexible component assembly.** The `EDAComponents` dataclass allows the user to compose any EDA from independently swappable components: seeding, selection, learning, sampling, replacement, mutation, crossover, local optimization, repairing, statistics, and stopping condition.
- **Extensibility.** Every component type is defined as an abstract base class. Adding a new learning algorithm, selection scheme, or test function requires only the implementation of the corresponding abstract method.
- **Single and multi-objective support.** The fitness function may return a scalar or a vector. Multi-objective selection (Pareto non-dominated sorting, crowding distance, indicator-based), decomposition (MOEA/D), and archive management are provided.
- **Knowledge extraction.** A dedicated module provides tools for analyzing the structure of the learned models, computing dependency measures, and visualizing network structures.

The remainder of this guide is organized as follows. Section 2 describes installation and the package structure. Section 3 explains how to define and execute a generic EDA using the core `EDA` class and `EDAComponents`. Section 4 surveys the plug-and-play algorithm classes. Section 5 details the probabilistic model families, organized by algorithm family. Section 6 documents all EDA component types and their implementations. Section 7 lists the built-in benchmark functions and test problems. Section 8 covers the multi-objective infrastructure. Section 9 describes the knowledge-extraction module. Section 10 explains mechanisms for injecting a-priori structural information.

2 Installation and Package Structure

2.1 Installation

pateda is distributed as a Python package located in the `packages/pateda` subdirectory of the repository. Install it in editable mode using:

```
1 pip install -e packages/pateda
```

Required dependencies: `numpy`, `scipy`, `scikit-learn`, `networkx`, `matplotlib`. Optional: `seaborn` (visualization).

2.2 Package Structure

After installation, the top-level `pateda` namespace provides direct access to the most commonly used classes:

```
1 from pateda import (  
2     EDA, EDAComponents,          # core  
3     UMDA, TreeEDA, EBNA, BOA,    # discrete plug-and-play  
4     MNFDA, MNFDAG, MOA, FDA,     # Markov network / FDA  
5     MKEDA, MTED, AffEDA,         # chain / mixture / affinity  
6     CUMDA, CFDA, PBIL, BSC,      # constrained / incremental  
7     GaussianUMDA, GaussianFull,  # continuous  
8     GaussianNetworkEDA,         # Gaussian network  
9     MixtureGaussianEDA,         # mixture of Gaussians  
10 )
```

The internal module tree is:

Module	Contents
<code>core/</code>	EDA executor, <code>EDAComponents</code> , <code>Statistics</code> , <code>Cache</code> , abstract component interfaces, model containers
<code>algorithms/</code>	Plug-and-play wrappers: <code>discrete.py</code> , <code>continuous.py</code> , <code>base.py</code>
<code>learning/</code>	All learning method implementations
<code>sampling/</code>	All sampling method implementations
<code>selection/</code>	Selection operators
<code>replacement/</code>	Replacement operators
<code>seeding/</code>	Population initialization methods
<code>mutation/</code>	Mutation operators
<code>crossover/</code>	Crossover operators
<code>repairing/</code>	Constraint repair methods
<code>local_optimization/</code>	Local search methods
<code>inference/</code>	MAP and k-MAP inference
<code>multiobjective/</code>	Dominance, crowding, MOEA/D, indicators, archive
<code>stop_conditions/</code>	Stopping criteria
<code>statistics/</code>	Population statistics and tracker
<code>functions/</code>	Benchmark functions (discrete and continuous)
<code>knowledge_extraction/</code>	Dependency analysis, network measures, visualizations

2.3 Running the Examples

The `packages/pateda/examples/` directory contains runnable scripts for all major algorithm families. For example:

```

1 cd packages/pateda/examples
2 python umda_onemax.py
3 python ebna_deceptive.py
4 python gaussian_umdaspHERE.py
5 python tree_eda_ising.py
6 python multiobjective_approaches_demo.py

```

3 Defining and Executing an EDA

3.1 General Description

The main component of pateda is the EDA class, defined in `core/eda.py`. It implements the generic EDA loop described in Algorithm 1, analogous to `RunEDA.m` in MATEDA-2.0.

Algorithm 1 Estimation of Distribution Algorithm

```

1: Set  $t \leftarrow 0$ 
2: repeat
3:   if  $t = 0$  then
4:     Generate initial population  $D_0$  using the seeding method
5:     Apply repairing method to  $D_0$  (if required)
6:     Evaluate  $D_0$  using the fitness function
7:     Apply local optimization to  $D_0$  (if required)
8:   else
9:     Sample  $D_{\text{new}}$  from the model using the sampling method
10:    Apply mutation to  $D_{\text{new}}$  (if required)
11:    Apply repairing method to  $D_{\text{new}}$  (if required)
12:    Evaluate  $D_{\text{new}}$  using the fitness function
13:    Apply local optimization to  $D_{\text{new}}$  (if required)
14:    Form  $D_t$  from  $D_{t-1}$  and  $D_{\text{new}}$  using the replacement method
15:  end if
16:  Select  $D_t^S \subseteq D_t$  using the selection method
17:  Optionally weight individuals in  $D_t^S$  (selection weighting)
18:  Learn a probabilistic model  $M_t$  from  $D_t^S$  using the learning method
19:   $t \leftarrow t + 1$ 
20: until the stopping condition is satisfied

```

3.2 The EDA Class

An EDA is instantiated with:

```

1 from pateda.core.eda import EDA
2
3 eda = EDA(
4     pop_size      = 300,
5     n_vars        = 20,
6     fitness_func  = my_func,
7     cardinality   = np.full(20, 2),    # binary variables
8     components    = my_components,
9     random_seed   = 42,
10 )
11 stats, cache = eda.run(verbose=True)

```

3.2.1 Input Parameters

pop_size Population size (integer).

n_vars Number of variables (integer).

fitness_func Callable that maps a 1-D NumPy array (an individual) to a scalar (single objective) or a 1-D array (multi-objective). `pateda` maximizes fitness; for minimization, negate the function.

cardinality For discrete problems: a 1-D integer array of length `n_vars` giving the number of values each variable can take. For continuous problems: a 2-D array of shape `(2, n_vars)` where row 0 contains lower bounds and row 1 upper bounds.

components An `EDAComponents` object specifying all the EDA building blocks (see Section 6).

random_seed Integer seed passed to NumPy's `default_rng`. Pass `None` for non-reproducible runs.

selection_weighting How the selected individuals are weighted when passed to the learning method. Options: `"uniform"` (default, equivalent to classic EDA), `"proportional"` (fitness proportional), `"boltzmann"` (Boltzmann distribution with temperature `weighting_beta`).

3.2.2 Output: Statistics and Cache

`eda.run()` returns a pair (`stats`, `cache`).

The `Statistics` object provides:

- `stats.best_fitness`: list of best fitness per generation.
- `stats.mean_fitness`: list of mean fitness per generation.
- `stats.std_fitness`: list of standard deviation per generation.
- `stats.best_individual`: the best solution found overall.
- `stats.best_fitness_overall`: its fitness value.
- `stats.generation_found`: the generation at which it was found.

The `Cache` object stores per-generation data controlled by a `CacheConfig`:

- `cache.populations`: list of population arrays.
- `cache.fitness_values`: list of fitness arrays.
- `cache.models`: list of learned models.
- `cache.selected_populations`: list of selected subsets.

3.3 The `EDAComponents` Dataclass

The `EDAComponents` dataclass (defined in `core/components.py`) specifies all building blocks of the EDA. All fields except `learning`, `sampling`, and `stop_condition` are optional.

```

1 from pateda.core.components import EDAComponents
2 from pateda.seeding.random_init import RandomInit
3 from pateda.selection.truncation import TruncationSelection
4 from pateda.learning.umdacore import LearnUMDA
5 from pateda.sampling.fda import SampleFDA
6 from pateda.replacement.elitist import ElitistReplacement
7 from pateda.stop_conditions.max_generations import MaxGenerations
8
9 components = EDAComponents(
10     seeding = RandomInit(),
11     selection = TruncationSelection(ratio=0.5),
12     learning = LearnUMDA(alpha=0.0),
13     sampling = SampleFDA(n_samples=300),
14     replacement = ElitistReplacement(n_elite=1),
15     stop_condition = MaxGenerations(100),
16 )

```

3.4 Example: UMDA on the OneMax Problem

```

1 import numpy as np
2 from pateda.core.eda import EDA
3 from pateda.core.components import EDAComponents
4 from pateda.seeding.random_init import RandomInit
5 from pateda.selection.truncation import TruncationSelection
6 from pateda.learning.umdacore import LearnUMDA
7 from pateda.sampling.fda import SampleFDA
8 from pateda.replacement.elitist import ElitistReplacement
9 from pateda.stop_conditions.max_generations_or_optimum import (
10     MaxGenerationsOrOptimum)
11
12 n_vars = 50
13 pop_size = 300
14
15 def onemax(x):
16     return np.sum(x)
17
18 cardinality = np.full(n_vars, 2) # binary variables
19
20 components = EDAComponents(
21     seeding = RandomInit(),
22     selection = TruncationSelection(ratio=0.5),
23     learning = LearnUMDA(alpha=0.0),
24     sampling = SampleFDA(n_samples=pop_size),
25     replacement = ElitistReplacement(n_elite=1),
26     stop_condition = MaxGenerationsOrOptimum(
27         max_gen=200, optimum=float(n_vars)),
28 )
29
30 eda = EDA(pop_size, n_vars, onemax, cardinality,
31           components, random_seed=42)
32 stats, cache = eda.run()
33 print("Best fitness:", stats.best_fitness_overall)
34 print("Best solution:", stats.best_individual)

```

3.5 Example: Gaussian UMDA on the Sphere Function

```

1 import numpy as np
2 from pateda import GaussianUMDA
3
4 def sphere(x):
5     return -np.sum(x**2)      # maximise the negated sphere
6
7 alg = GaussianUMDA(
8     n_vars      = 30,
9     bounds      = (-5.0, 5.0),
10    fitness_func= sphere,
11    pop_size     = 500,
12    n_gen        = 100,
13    random_seed  = 0,
14 )
15 stats, _ = alg.run()
16 print(f"Best: {stats.best_fitness_overall:.6f}")

```

3.6 Fitness Convention

pateda **maximizes** fitness. When the objective is to be minimized (e.g., energy, error, distance), the user should negate the function:

```

1 def minimization_objective(x):
2     return -cost_function(x)      # pateda maximises this

```

4 Plug-and-Play Algorithm Classes

For convenience, pateda provides ready-to-use wrappers that assemble the appropriate components internally. These classes expose a minimal constructor and a `run()` method that returns (stats, cache).

4.1 Discrete Algorithm Classes

All discrete algorithm classes share the following common parameters:

Parameter	Description
<code>n_vars</code>	Number of variables
<code>cardinality</code>	int (same for all vars) or 1-D integer array
<code>fitness_func</code>	Callable; higher is better
<code>pop_size</code>	Population size (default: 100)
<code>n_gen</code>	Number of generations (default: 50)
<code>selection_ratio</code>	Truncation fraction (default: 0.5)
<code>elitism</code>	Preserve best individual (default: True)
<code>random_seed</code>	RNG seed (default: None)
<code>alpha</code>	Laplace smoothing for probability tables (default: 0.0)

Table 1 summarizes the discrete algorithm classes.

4.2 Continuous Algorithm Classes

Table 2 summarizes the continuous algorithm classes.

Common parameters for continuous classes are `n_vars`, `bounds` (a tuple (lo, hi) or a (2, n_vars) array), `fitness_func`, `pop_size`, `n_gen`, `selection_ratio`, `elitism`, and `random_seed`.

Class	Identifier	Description
UMDA	UMDA	Univariate marginal distribution algorithm (Mühlenbein and Paaß, 1996)
PBIL	PBIL	Population-based incremental learning (Baluja, 1994)
BSC	BSC	Fitness-weighted univariate marginals (Bisection scheme (Inza et al., 2000))
MIMIC	MIMIC	Mutual-information maximization for input clustering (De Bonet et al., 1997)
BMDA	BMDA	Bivariate marginal distribution algorithm
TreeEDA	Tree-EDA	Maximum-weight spanning tree of pairwise MI (Baluja and Davies, 1997; Pelikan and Mühlenbein, 1999)
TreeEDAR	Tree-EDA_r	Tree-EDA with restricted interaction matrix
AffEDA	AffEDA	Affinity-propagation cliques (Frey and Dueck, 2007; Santana et al., 2008a)
MKEDA	MK-EDA _k	<i>k</i> -order Markov chain EDA (Santana et al., 2004)
MTED	MT-EDA	Mixture of trees EDA (Santana et al., 2001)
MNFDA	MN-FDA	Markov network FDA sampling (Santana, 2005)
MNFDAR	MN-FDA_r	MN-FDA with random variable ordering
MNFDAG	MN-FDAg	Augmented Markov network + Gibbs sampling
MNFDAGR	MN-FDAg_r	MN-FDAg with random Gibbs ordering
MOA	MOA	Markovianity-based optimization algorithm (Shakya and McCall, 2007; Shakya and Santana, 2008)
EBNA	EBNA	Estimation of Bayesian network algorithm (Etzeberria and Larrañaga, 1999)
BOA	BOA	Bayesian optimization algorithm (Pelikan et al., 1999; Pelikan, 2005)
FDA	FDA	Factorized distribution algorithm (Mühlenbein et al., 1999)
CUMDA	CUMDA	Unitation-constrained UMDA
CFDA	CFDA	Unitation-constrained FDA

Table 1: Discrete plug-and-play algorithm classes in pateda.

4.3 Quick Comparison Example

```

1 from pateda import UMDA, TreeEDA, EBNA
2 import numpy as np
3
4 def deceptive3(x):
5     """Sum of order-3 deceptive functions."""
6     n = len(x)
7     score = 0.0
8     for i in range(0, n - 2, 3):
9         u = int(x[i]) + int(x[i+1]) + int(x[i+2])
10        score += 0.9 if u < 3 else 1.0
11    return score
12
13 n_vars = 30
14 for AlgClass in [UMDA, TreeEDA, EBNA]:

```

Class	Identifier	Description
GaussianUMDA	G-UMDA	Univariate Gaussian marginals (Larrañaga et al., 1999, 2000)
GaussianFull	G-Full	Full multivariate Gaussian (Larrañaga et al., 2000; Bosman and Thierens, 2000b)
GaussianNetworkEDA	G-Net	Gaussian network (GMRF) EDA
MixtureGaussianEDA	MGaussian	Mixture of Gaussians (Bosman and Thierens, 2002)

Table 2: Continuous plug-and-play algorithm classes in pateda.

```

15 alg = AlgClass(n_vars=n_vars, cardinality=2,
16               fitness_func=deceptive3,
17               pop_size=500, n_gen=100, random_seed=1)
18 stats, _ = alg.run(verbose=False)
19 print(f"{AlgClass.__name__:10s} best={stats.best_fitness_overall
      :.3f}")

```

5 Probabilistic Models and Algorithm Families

5.1 Discrete Representation

Discrete variables take values in $\{0, 1, \dots, r_i - 1\}$ where r_i is the cardinality of variable X_i . Binary variables have $r_i = 2$. pateda represents a population of N solutions as a 2-D NumPy array of shape (N, n) with integer dtype. The `cardinality` parameter is a 1-D integer array $[r_1, r_2, \dots, r_n]$. Mixed-cardinality problems (some variables binary, others multi-valued) are fully supported.

5.2 Continuous Representation

Continuous variables are real-valued, bounded by $[\ell_i, u_i]$. The population is a 2-D float array of shape (N, n) . The `cardinality` array for continuous problems is a 2-D array of shape $(2, n)$: row 0 contains the lower bounds and row 1 the upper bounds.

5.3 Factorized Distributions

A factorized distribution (Mühlenbein et al., 1999; Harik, 1999) represents a joint probability over $\mathbf{X} = (X_1, \dots, X_n)$ as a product of factor (clique) marginals:

$$p(\mathbf{x}) = \prod_{i=1}^m p(x_{C_i} \mid x_{S_i}) \quad (1)$$

where C_i is the set of “new” variables in factor i and S_i is the set of “separator” variables shared with previous factors. This class of models, called *ordered factorizations* (Santana, 2005), subsumes:

- Marginal product factorizations (univariate, bivariate, etc.)
- Markov chain factorizations
- Junction-tree factorizations
- Junction-graph factorizations (Santana et al., 2008b)

pateda stores factorizations in the **FactorizedModel** container, which holds a list of factors; each factor records its clique variables, separator variables, and a conditional probability table. The **LearnFDA** class fits the tables from a given population, and **SampleFDA** samples new solutions using probabilistic logic sampling (PLS) (Mühlenbein and Paaß, 1996).

FDA (FDA class): when a user supplies a fixed clique structure (known from the problem domain), **LearnFDA** computes only the conditional probability tables from data, bypassing structure learning entirely. This is the factorized distribution algorithm of Mühlenbein et al. (1999).

5.4 Univariate Models

Univariate models assume all variables are independent: $p(\mathbf{x}) = \prod_i p(x_i)$. The marginal $p(x_i)$ is estimated as the empirical frequency in the selected population (with optional Laplace smoothing α).

UMDA (Mühlenbein and Paaß, 1996). Univariate marginal distribution algorithm. Each marginal is learned independently from the selected population. Learning: **LearnUMDA**. Sampling: **SampleFDA**.

PBIL (Baluja, 1994). Population-based incremental learning. Maintains a running estimate of the marginals updated with learning rate α : $P^{(t+1)}(x_i) = (1 - \alpha) P^{(t)}(x_i) + \alpha \hat{P}(x_i)$. With $\alpha = 1$ PBIL reduces to UMDA. Learning: **LearnPBIL**. Sampling: **SampleFDA**.

BSC (Inza et al., 2000). Bisection-based fitness-weighted marginals. $P(x_i = k) \propto \sum_{\{j: x_i^{(j)} = k\}} f(x^{(j)})$. Learning: **LearnBSC**. Sampling: **SampleFDA**.

MIMIC (De Bonet et al., 1997). Mutual Information Maximization for Input Clustering. Learns a chain-structured factorization that maximizes the sum of pairwise mutual information along the chain. Learning: **LearnMIMIC**. Sampling: **SampleFDA**.

5.5 Tree and Forest Models

Tree-structured models represent pairwise variable dependencies using a maximum-weight spanning tree of the mutual-information matrix (Chow and Liu, 1968).

TreeEDA / **Tree-EDA** (Baluja and Davies, 1997; Pelikan and Mühlenbein, 1999). Learns a directed tree using Chow–Liu algorithm on pairwise MI. Learning: **LearnTreeModel**. Sampling: **SampleFDA**.

TreeEDAR / **Tree-EDA_r** Variant of Tree-EDA with a user-supplied **interaction_matrix** that constrains which pairs of variables can form a dependency. Learning: **LearnTreeModelR**.

BMDA Bivariate marginal distribution algorithm. Detects pairwise dependencies with chi-square tests and builds a forest of bivariate conditionals. Learning: **LearnBMDA**.

AffEDA Affinity-propagation EDA (Frey and Dueck, 2007; Santana et al., 2008a). Uses affinity propagation on the MI matrix to discover variable cliques, then builds a factorized model over those cliques. Learning: **LearnAffinityFactorization**.

MTED / **MT-EDA** (Santana et al., 2001). Mixture of trees EDA. Learns k tree components with mixture weights. Weight learning options: "uniform", "em", or "fitness_proportional". Adaptive and prior-based extensions are supported to prevent premature convergence. Learning: **LearnMixtureTrees**. Sampling: **SampleMixtureTrees**.

5.6 Bayesian Network Models

Bayesian networks (Lauritzen and Spiegelhalter, 1988) are directed acyclic graphs in which X_i is conditionally independent of its non-descendants given its parents $\mathbf{Pa}_i$. The joint distribution factorizes as:

$$p(\mathbf{x}) = \prod_{i=1}^n p(x_i \mid \mathbf{pa}_i). \quad (2)$$

The local distribution $p(x_i \mid \mathbf{pa}_i)$ is a conditional probability table (CPT) with r_i entries for each of the $q_i = \prod_{X_j \in \mathbf{Pa}_i} r_j$ parent configurations.

EBNA (Etzeberria and Larrañaga, 1999). Estimation of Bayesian network algorithm. Structure learning uses a greedy score-and-search procedure with the Bayesian information criterion (BIC) (Schwarz, 1978) or the Bayesian score (Cooper and Herskovits, 1992). Learning: `LearnEBNA`. Sampling: `SampleBayesianNetwork`.

BOA (Pelikan et al., 1999; Pelikan, 2005). Bayesian optimization algorithm. Greedy structure search with BIC/MDL scoring. Learning: `LearnBOA`. Sampling: `SampleBayesianNetwork`.

Both EBNA and BOA sample new individuals using probabilistic logic sampling over the directed network. Additionally, pateda provides sampling strategies that find the *most probable explanation* (MPE): the configuration $\mathbf{x}^* = \arg \max_{\mathbf{x}} p(\mathbf{x})$, computed via `SampleGibbs` or the inference methods in `inference/map_inference.py`.

5.7 Markov Network Models

A Markov random field (MRF) (Hammersley and Clifford, 1968) is an undirected graphical model in which the joint distribution factorizes over the cliques of the graph as a Gibbs distribution:

$$p(\mathbf{x}) = \frac{1}{Z} \exp(-H(\mathbf{x})), \quad H(\mathbf{x}) = \sum_{C \in \mathcal{C}} \Phi_C(\mathbf{x}_C) \quad (3)$$

where $\mathcal{C}$ is the set of maximal cliques and Z is the partition function. Sampling from MRFs typically uses Gibbs sampling: iteratively sample each variable X_i from its conditional distribution given the current values of its neighbors $N(X_i)$.

MN-FDA / MNFDA (Santana, 2005). Learns a Markov network structure from chi-square independence tests between pairs of variables and represents the joint distribution as a factorized model. Sampling uses PLS (factorized distribution sampling). Learning: `LearnMNFDA`. Sampling: `SampleFDA`.

MN-FDA_r / MNFDAR Variant of MN-FDA with random variable ordering during sampling, improving exploration of multimodal distributions. Learning: `LearnMNFDAR`. Sampling: `SampleFDA`.

MN-FDAg / MNFDAG Augmented Markov network EDA. Extends MN-FDA with additional edges based on mutual information. Sampling uses Gibbs sampling. Learning: `LearnMNFDAG`. Sampling: `SampleGibbs`.

MN-FDAg_r / MNFDAGR MN-FDAg with random Gibbs sampling order. Learning: `LearnMNFDAGR`. Sampling: `SampleGibbs`.

MOA (Shakya and McCall, 2007; Shakya and Santana, 2008). Markovianity-based optimization algorithm. Learns local k -neighborhoods for each variable and samples via Gibbs sampling. Learning: `LearnMOA`. Sampling: `SampleGibbs`.

5.8 Markov Chain Models

MK-EDA / MKEDA (Santana et al., 2004). k -order Markov chain EDA. Each variable X_i depends on the k preceding variables in a fixed linear ordering: $p(\mathbf{x}) = p(x_1) \prod_{i=2}^n p(x_i | x_{i-k}, \dots, x_{i-1})$. Well-suited to problems with a natural sequential structure (e.g. protein folding with relative encodings). Learning: `LearnMarkovChain`. Sampling: `SampleMarkovChain`.

5.9 Constrained EDAs

CUMDA Unitation-constrained UMDA. Every solution must contain exactly `n_ones` ones. The seeding method (`SeedingUnitationConstraint`) generates feasible initial solutions, and the sampling method (`SampleCUMDA`) uses Stochastic Universal Sampling to maintain the constraint. Learning: `LearnCUMDA`.

CFDA Unitation-constrained FDA. Uses a tree-structured factorized model while enforcing the same fixed-union constraint as CUMDA. Learning: `LearnCFDA`. Sampling: `SampleCFDA`.

5.10 Continuous Gaussian Models

GaussianUMDA (Larrañaga et al., 1999, 2000). Each variable is modeled independently as a Gaussian: $p(x_i) = \mathcal{N}(x_i; \mu_i, \sigma_i^2)$. Parameters (μ_i, σ_i^2) are estimated from the selected population. Learning: `LearnGaussianUnivariate`. Sampling: `SampleGaussianUnivariate`.

GaussianFull (Larrañaga et al., 2000; Bosman and Thierens, 2000b). Full multivariate Gaussian: $p(\mathbf{x}) = \mathcal{N}(\mathbf{x}; \boldsymbol{\mu}, \Sigma)$. All pairwise dependencies are captured by the covariance matrix Σ . May suffer from variance collapse in later generations (Bosman and Grahl, 2008); a variance scaling parameter is provided. Learning: `LearnGaussianFull`. Sampling: `SampleGaussianFull`.

GaussianNetworkEDA Gaussian network (GMRF) EDA. Learns a sparse Gaussian graphical model where each variable's local distribution is a linear regression on its parents in a directed graph. The GMRF approach identifies relevant conditional independences in continuous data.

MixtureGaussianEDA (Bosman and Thierens, 2002). Mixture of Gaussian distributions. The population is first clustered (k-means or similar), then a separate Gaussian (univariate or full) is fit to each cluster. Suitable for multimodal search landscapes. Learning: `LearnMixtureGaussian`. Sampling: `SampleMixtureGaussian`.

6 EDA Components

6.1 Seeding Methods

Seeding methods initialize the population at generation 0.

RandomInit Samples each variable uniformly from its range. For discrete variables: integer in $[0, r_i)$. For continuous variables: real in $[\ell_i, u_i]$.

BiasInit Biased binary initialization. Variable X_i is set to 1 with probability p and to 0 with probability $1 - p$.

SeedingUnitationConstraint Generates binary solutions with exactly `num_ones` ones. Required by CUMDA and CFDA.

SeedThisPop Seeds the population with a user-supplied array.

6.2 Selection Methods

Selection determines which individuals are used for model learning. The role of selection in EDAs has been studied by Johnson and Shapiro (2002); Lima et al. (2007); Mahnig and Mühlenbein (2001); Santana et al. (2005).

TruncationSelection Selects the top $\lfloor \text{ratio} \cdot N \rfloor$ individuals by fitness. This is the most common selection method in EDAs.

TournamentSelection Tournament selection of configurable size.

ProportionalSelection / SUSSelection Fitness-proportional selection using stochastic universal sampling (SUS) (Baker, 1987).

BoltzmannSelection Selection with Boltzmann temperature; probability proportional to $e^{\beta f(x)}$.

RankingSelection Rank-based selection; probability proportional to $2r/(N(N+1))$ where r is the fitness rank.

NonDominatedSelection Selects the non-dominated front. For multi-objective problems.

CrowdingSelection NSGA-II-style: non-dominated sorting + crowding distance tie-breaking.

IndicatorBasedSelection SMS-EMOA-style hypervolume indicator-based selection.

ParetoFrontSelection Selects complete Pareto fronts up to the selection quota.

6.3 Replacement Methods

ElitistReplacement Copies the best `n_elite` solutions from the previous generation into the new population if they are better than the worst new individual.

GenerationalReplacement New population completely replaces the old one.

6.4 Stopping Conditions

MaxGenerations Stops after a fixed number of generations.

MaxGenerationsOrOptimum Stops when either the generation limit is reached or a target fitness value is attained.

6.5 Local Optimization

GreedySearch Greedy hill climbing in the continuous case.

DiscreteGreedySearch Greedy flip-based local search for discrete problems.

DiscreteSimulatedAnnealing Simulated annealing for discrete problems.

ContiguousBlockOptimization Specialized local search for contiguous-block problems.

ScipyLocalSearch Wraps `scipy.optimize.minimize` for continuous local optimization.

6.6 Repairing Methods

BoundsRepair Clips continuous variable values to their feasible bounds.

TrigonometricRepair Trigonometric projection for angular variables.

UnitationRepair Adjusts binary solutions to satisfy a fixed unitation constraint.

7 Test Functions and Benchmark Problems

7.1 Discrete Functions

OneMax Maximizes $f(\mathbf{x}) = \sum_i x_i$. The simplest EDA benchmark; all variables are independent.

Trap functions Deceptive order- k subfunctions (Ackley, 1987): $f(\mathbf{x}) = \sum_j \text{trap}_k(u_j)$ where u_j is the number of ones in block j . Optimal solution is the all-ones string; the deceptive attractor is the all-zeros string.

Deceptive functions Goldberg’s deceptive functions (Goldberg, 1987).

Additively decomposable functions (ADFs) $f(\mathbf{x}) = \sum_i f_i(\mathbf{x}_{S_i})$ where each subfunction f_i is defined over a disjoint subset S_i of variables. Ideal for studying the structural identification capabilities of EDAs.

NK landscapes (Kauffman, 1993). Each variable depends on k others drawn uniformly or in circular topology. The subfunction value for each configuration is drawn at random.

Ising model Energy minimization on a lattice: $E = -\sum_{i<j} J_{ij}\sigma_i\sigma_j - \sum_i h_i\sigma_i$.

HP protein folding (Dill, 1985). 2-D/3-D lattice model. Solutions encode the sequence of relative moves of amino acid residues. pateda uses the relative encoding with a backtracking repair heuristic.

SAT (Selman et al., 1992). Satisfiability problems; single- and multi-objective versions available.

UBQP Unconstrained binary quadratic programming: $f(\mathbf{x}) = \mathbf{x}^T Q \mathbf{x}$.

Contiguous-block functions Reward solutions with long contiguous blocks of identical bits.

Integer functions Functions over multi-valued (non-binary) discrete variables.

7.2 Continuous Functions

Standard benchmarks (`functions/continuous/benchmarks.py`): Sphere, Rastrigin, Rosenbrock, Ackley, Griewank.

AB off-lattice protein model (Larrañaga et al., 1999). Off-lattice protein model with Lennard-Jones interactions. Variables are bond angles in $[0, 2\pi]$.

GNBG instances Generalized numerical benchmark generator instances (`GNBG_Instances.Python-main/`).

8 Multi-Objective Optimization

8.1 Multi-Objective Problem Definition

A multi-objective optimization problem (MOP) (Deb, 2001) requires simultaneously optimizing m objectives $f_1(\mathbf{x}), \dots, f_m(\mathbf{x})$. Since objectives usually conflict, the goal is to find the *Pareto front*: the set of non-dominated solutions where no single objective can be improved without degrading another.

pateda supports multi-objective problems by returning a vector from the fitness function:

```
1 def bi_objective(x):
2     f1 = np.sum(x) # first objective (maximise)
3     f2 = -np.sum((x - 0.5)**2) # second objective (maximise)
4     return np.array([f1, f2])
```

8.2 Pareto Dominance

pateda implements Pareto dominance in `multiobjective/dominance.py`. Solution $\mathbf{x}$ dominates $\mathbf{y}$ if it is at least as good on all objectives and strictly better on at least one.

8.3 Multi-Objective Selection

Non-dominated sorting + crowding (NSGA-II) (Deb et al., 2002). Solutions are ranked into Pareto fronts. Within each front, crowding distance breaks ties. Available via `CrowdingSelection`.

Indicator-based (SMS-EMOA) Hypervolume contribution per individual. Available via `IndicatorBasedSelection` and `multiobjective/indicators.py`.

Decomposition (MOEA/D) (Mühlenbein et al., 1999). Decomposes the MOP into scalar subproblems via weight vectors. pateda's MOEA/D is implemented in `multiobjective/moad.py`.

8.4 Multi-Objective EDA Pipeline

Below is a sketch of a multi-objective EDA using NSGA-II-style selection with a Bayesian network learner:

```
1 from pateda.core.eda import EDA
2 from pateda.core.components import EDAComponents
3 from pateda.seeding.random_init import RandomInit
4 from pateda.selection.crowding import CrowdingSelection
5 from pateda.learning.ebna import LearnEBNA
6 from pateda.sampling.bayesian_network import SampleBayesianNetwork
7 from pateda.replacement.elitist import ElitistReplacement
8 from pateda.stop_conditions.max_generations import MaxGenerations
9 import numpy as np
10
11 n_vars, pop_size = 20, 500
12
13 def mo_func(x):
14     f1 = np.sum(x)
15     f2 = np.sum(1 - x)
16     return np.array([f1, f2])
17
18 components = EDAComponents(
19     seeding = RandomInit(),
20     selection = CrowdingSelection(n_select=pop_size // 2),
21     learning = LearnEBNA(),
22     sampling = SampleBayesianNetwork(n_samples=pop_size),
23     replacement = ElitistReplacement(n_elite=1),
24     stop_condition = MaxGenerations(50),
25 )
26 eda = EDA(pop_size, n_vars, mo_func,
27           np.full(n_vars, 2), components)
28 stats, cache = eda.run()
```

8.5 Extracting Pareto Fronts

After running the EDA with `cache_config` that stores populations and fitness values, the full Pareto set approximation can be extracted:

```
1 from pateda.selection.utils.pareto import find_pareto_front
2 import numpy as np
3
```



```

4 all_pop = np.vstack(cache.populations)
5 all_fit = np.vstack(cache.fitness_values)
6 pareto_idx = find_pareto_front(all_fit)
7 pareto_pop = all_pop[pareto_idx]
8 pareto_fit = all_fit[pareto_idx]

```

9 Knowledge Extraction

One of the most distinctive features of pateda—inherited from MATEDA-2.0—is the ability to extract and visualize knowledge from the probabilistic models learned during the EDA search. The `knowledge_extraction` module provides tools organized into the following categories.

9.1 Dependency Analysis

`knowledge_extraction/dependency_analysis.py` provides:

compute_correlation_matrix Computes the Pearson correlation matrix between variables from a population.

learn_bayesian_network Learns a Bayesian network structure from a population using score-and-search.

learn_gaussian_network Learns a Gaussian graphical model from continuous data.

analyze_variable_dependencies Computes pairwise mutual information and tests for marginal independence.

compute_mutual_information Mutual information between two discrete random variables.

9.2 Fitness Measures

`knowledge_extraction/fitness_measures.py` provides tools to quantify the EDA’s behavior over generations:

Response to selection $R(t)$ $R(t) = \bar{f}(t+1) - \bar{f}(t)$: improvement in mean fitness from one generation to the next.

Amount of selection $S(t)$ $S(t) = \bar{f}_s(t) - \bar{f}(t)$: difference between the mean fitness of the selected subset and the whole population.

Realized heritability $b(t)$ $b(t) = R(t)/S(t)$: fraction of the selection differential that is realized in the next generation (Mühlenbein, 1997).

9.3 Network Measures and Visualizations

For models based on graphs (Bayesian networks, Markov networks, trees), pateda tracks which edges appear across generations and runs:

Edge frequency matrix (Echegoyen et al., 2007; Santana et al., 2007). The (i, j) entry counts how often the edge (X_i, X_j) appeared in the structures learned across all runs and generations. Bright colors in the heat-map indicate frequent structural relationships, often corresponding to true problem-structure interactions.

Dendrogram of edges Hierarchical clustering of edges by co-occurrence frequency, displayed as a dendrogram (Hauschild and Pelikan, 2008).

Parallel coordinate view Each axis is a selected edge; each line connects edges that co-occur in the same structure at the same generation.

Glyph representation A grid of glyphs (one per run $\times$ generation) showing which edges are present in each learned structure.

9.4 Continuous Variable Analysis

For continuous EDAs, `knowledge_extraction/continuous_visualizations.py` and `gaussian_networks.py` provide tools to plot the evolution of Gaussian parameters (means, variances, correlations) across generations. The `vine_analysis.py` module supports analysis of vine copula structures.

10 Injecting A-Priori Knowledge

A key advantage of EDA-based approaches is that a-priori information about the problem structure can be directly incorporated into the model, potentially reducing the search space and accelerating convergence (Santana et al., 2009).

10.1 Fixed Factorizations

When the structure of the optimal model is known a priori (e.g., from domain knowledge or problem decomposition), the user can supply a fixed clique structure to FDA:

```

1 import numpy as np
2 from pateda import FDA
3
4 # Fixed Markov-chain structure for a 10-variable problem
5 n = 10
6 cliques = np.array([
7     [0, 1, 0],      # factor 1: only X_1
8     [1, 1, 0, 1],   # factor 2: X_1 / X_2
9     [1, 1, 1, 2],   # factor 3: X_2 / X_3
10    # ... etc.
11 ])
12
13 alg = FDA(n_vars=n, cardinality=2,
14          fitness_func=my_func,
15          cliques=cliques,
16          pop_size=300, n_gen=50)
17 stats, _ = alg.run()
```

The Markov-chain FDA with order 1 is the natural model for problems with sequential structure, such as the HP protein folding model (Dill, 1985; Santana et al., 2004).

10.2 Interaction Matrices

TreeEDAR, MNFDAR, and MNFDAGR accept an `interaction_matrix` parameter: an $n \times n$ binary matrix where entry $(i, j) = 1$ means that variables X_i and X_j may be connected in the learned model, and $(i, j) = 0$ forbids the connection. This allows the user to encode known independence relationships or restrict the model to biologically or physically plausible interactions.

10.3 Biased Initialization

BiasInit seeds the initial population with variable-wise biases. For example, if domain knowledge indicates that variable X_5 is likely to take value 1 in good solutions, a higher prior probability can be assigned:

```

1 from pateda.seeding.bias_init import BiasInit
2
3 seeder = BiasInit(probs=np.array([0.5]*20 + [0.9]))
4 # Variable 21 is initialized with  $P(X=1) = 0.9$ 

```

10.4 Constrained Search

CUMDA and CFDA enforce a fixed number of ones in every solution. This is useful for combinatorial problems where a cardinality constraint must be satisfied throughout the search (e.g., feature selection with exactly k features).

10.5 Selection Weighting

The `selection_weighting` parameter of the EDA constructor allows fitness-proportional or Boltzmann weighting of the selected individuals when computing model parameters, effectively blending structure learning with fitness guidance:

```

1 eda = EDA(pop_size, n_vars, fitness_func, cardinality,
2           components,
3           selection_weighting="boltzmann",
4           weighting_beta=2.0) # higher beta      stronger fitness
                               pressure

```

11 Conclusions

pateda is a comprehensive Python library for estimation of distribution algorithms supporting:

- Discrete and continuous optimization, single and multi-objective.
- A broad range of EDA families: univariate, tree, Bayesian network, Markov network, factorized, mixture, Gaussian, constrained.
- Flexible component assembly via `EDAComponents`, enabling rapid prototyping of novel EDA variants.
- Mechanisms for incorporating a-priori problem structure knowledge.
- A knowledge-extraction module for model analysis and visualization.
- A rich library of benchmark functions including classical combinatorial problems, continuous benchmarks, and real-world problem instances.

Future versions will extend the guide with detailed worked examples for each algorithm family, additional multi-objective benchmarks (DTLZ, ZDT), performance comparison tables, and deeper documentation of the knowledge-extraction and visualization tools.

References

- Ackley, D. H. (1987). An empirical study of bit vector function optimization. *Genetic Algorithms and Simulated Annealing*, pages 170–204.
- Baker, J. E. (1987). Reducing bias and inefficiency in the selection algorithm. In *Proceedings of the Second International Conference on Genetic Algorithms*, pages 14–21. Lawrence Erlbaum Associates (Hillsdale).

- Baluja, S. (1994). Population-based incremental learning: A method for integrating genetic search based function optimization and competitive learning. Technical Report CMU-CS-94-163, Carnegie Mellon University, Pittsburgh, PA.
- Baluja, S. and Davies, S. (1997). Using optimal dependency-trees for combinatorial optimization: Learning the structure of the search space. In Fisher, D. H., editor, *Proceedings of the 14th International Conference on Machine Learning*, pages 30–38, San Francisco, CA. Morgan Kaufmann.
- Bosman, P. A. and Grahl, J. (2008). Matching inductive search bias and problem structure in continuous estimation of distribution algorithms. *European Journal of Operational Research*, 185:1246–1264.
- Bosman, P. A. and Thierens, D. (2000a). Expanding from discrete to continuous estimation of distribution algorithms: The IDEA. In *Parallel Problem Solving from Nature - PPSN VI 6th International Conference*, Paris, France. Springer. Lecture Notes in Computer Science 1917.
- Bosman, P. A. and Thierens, D. (2000b). IDEAs based on the normal kernels probability density function. Technical Report UU-CS-2000-11, Utrecht University.
- Bosman, P. A. and Thierens, D. (2002). Multi-objective optimization with diversity preserving mixture-based iterated density estimation evolutionary algorithms. *International Journal of Approximate Reasoning*, 31(3):259–289.
- Chow, C. K. and Liu, C. N. (1968). Approximating discrete probability distributions with dependence trees. *IEEE Transactions on Information Theory*, 14(3):462–467.
- Cooper, G. F. and Herskovits, E. (1992). A Bayesian method for the induction of probabilistic networks from data. *Machine Learning*, 9:309–347.
- De Bonet, J. S., Isbell, C. L., and Viola, P. (1997). MIMIC: Finding optima by estimating probability densities. In Mozer, M. C., Jordan, M. I., and Petsche, T., editors, *Advances in Neural Information Processing Systems*, volume 9, pages 424–430. The MIT Press, Cambridge.
- Deb, K. (2001). *Multi-objective Optimization Using Evolutionary Algorithms*. Wiley, Chichester, UK.
- Deb, K., Pratap, A., Agarwal, S., and Meyarivan, T. (2002). A fast and elitist multiobjective genetic algorithm: NSGA-II. *IEEE Transactions on Evolutionary Computation*, 6(2):182–197.
- Dill, K. A. (1985). Theory for the folding and stability of globular proteins. *Biochemistry*, 24(6):1501–1509.
- Echegoyen, C., Lozano, J. A., Santana, R., and Larrañaga, P. (2007). Exact Bayesian network learning in estimation of distribution algorithms. In *Proceedings of the 2007 Congress on Evolutionary Computation CEC-2007*, pages 1051–1058. IEEE Press.
- Echegoyen, C., Santana, R., Lozano, J. A., and Larrañaga, P. (2008). The impact of probabilistic learning algorithms in EDAs based on Bayesian networks. In *Linkage in Evolutionary Computation*, Studies in Computational Intelligence, pages 109–139. Springer.
- Etzeberria, R. and Larrañaga, P. (1999). Global optimization using Bayesian networks. In Ochoa, A., Soto, M. R., and Santana, R., editors, *Proceedings of the Second Symposium on Artificial Intelligence (CIMA-99)*, pages 332–339. Editorial Academia. Havana, Cuba.
- Frey, B. J. and Dueck, D. (2007). Clustering by passing messages between data points. *Science*, 315:972–976.

- Goldberg, D. E. (1987). Simple genetic algorithms and the minimal, deceptive problem. In Davis, L., editor, *Genetic Algorithms and Simulated Annealing*, pages 74–88. Pitman Publishing, London, UK.
- Goldberg, D. E. (1989). *Genetic Algorithms in Search, Optimization, and Machine Learning*. Addison-Wesley, Reading, MA.
- Hammersley, J. M. and Clifford, P. (1968). Markov fields of finite graphs and lattice. Technical report, University of California-Berkeley.
- Harik, G. (1999). Linkage learning via probabilistic modeling in the ECGA. IlliGAL Report 99010, University of Illinois at Urbana-Champaign, Illinois Genetic Algorithms Laboratory, Urbana, IL.
- Hauschild, M. and Pelikan, M. (2008). Enhancing efficiency of hierarchical BOA via distance-based model restrictions. MEDAL Report No. 2008007, Missouri Estimation of Distribution Algorithms Laboratory (MEDAL).
- Holland, J. H. (1975). *Adaptation in Natural and Artificial Systems: An Introductory Analysis with Applications to Biology, Control, and Artificial Intelligence*. University of Michigan Press, Ann Arbor, MI.
- Inza, I., Larrañaga, P., Etxeberria, R., and Sierra, B. (2000). Feature subset selection by Bayesian network-based optimization. *Artificial Intelligence*, 123(1-2):157–184.
- Johnson, A. and Shapiro, J. L. (2002). The importance of selection mechanisms in distribution estimation algorithms. In Collet, P., editor, *Proceedings of EA 2001*, volume 2310 of *Lecture Notes in Computer Science*, pages 91–103. Springer.
- Kauffman, S. (1993). *Origins of Order*. Oxford University Press.
- Larrañaga, P., Etxeberria, R., Lozano, J. A., and Peña, J. (2000). Combinatorial optimization by learning and simulation of Bayesian networks. In *Proceedings of the Sixteenth Annual Conference on Uncertainty in Artificial Intelligence (UAI-2000)*, pages 343–352, San Francisco, CA. Morgan Kaufmann Publishers.
- Larrañaga, P., Etxeberria, R., Lozano, J. A., and Peña, J. M. (1999). Optimization by learning and simulation of Bayesian and Gaussian networks. Technical Report EHU-KZAA-IK-4/99, Department of Computer Science and Artificial Intelligence, University of the Basque Country.
- Larrañaga, P. and Lozano, J. A., editors (2002). *Estimation of Distribution Algorithms. A New Tool for Evolutionary Computation*. Kluwer Academic Publishers, Boston/Dordrecht/London.
- Lauritzen, S. and Spiegelhalter, D. J. (1988). Local computations with probabilities on graphical structures and their application to expert systems (with discussion). *Journal of the Royal Statistical Society, Series B*, 50:157–224.
- Lima, C. F., Pelikan, M., Goldberg, D. E., Lobo, F. G., Sastry, K., and Hauschild, M. (2007). Influence of selection and replacement strategies on linkage learning in BOA. In *Proceedings of the 2007 Congress on Evolutionary Computation CEC-2007*, pages 1083–1090. IEEE Press.
- Lozano, J. A., Larrañaga, P., Inza, I., and Bengoetxea, E., editors (2006). *Towards a New Evolutionary Computation: Advances on Estimation of Distribution Algorithms*. Springer.
- Mahnig, T. and Mühlenbein, H. (2001). Comparing the adaptive Boltzmann selection schedule SDS to truncation selection. In *Evolutionary Computation and Probabilistic Graphical Models. Proceedings of the Third Symposium on Adaptive Systems (ISAS-2001)*, pages 121–128, Havana, Cuba.

- Mühlenbein, H. (1997). The equation for response to selection and its use for prediction. *Evolutionary Computation*, 5(3):303–346.
- Mühlenbein, H., Mahnig, T., and Ochoa, A. (1999). Schemata, distributions and graphical models in evolutionary optimization. *Journal of Heuristics*, 5(2):213–247.
- Mühlenbein, H. and Paaß, G. (1996). From recombination of genes to the estimation of distributions I. Binary parameters. In Voigt, H.-M., Ebeling, W., Rechenberg, I., and Schwefel, H.-P., editors, *Parallel Problem Solving from Nature - PPSN IV*, volume 1141 of *Lectures Notes in Computer Science*, pages 178–187, Berlin. Springer.
- Pelikan, M. (2005). *Hierarchical Bayesian Optimization Algorithm. Toward a New Generation of Evolutionary Algorithms*, volume 170 of *Studies in Fuzziness and Soft Computing*. Springer.
- Pelikan, M., Goldberg, D. E., and Cantú-Paz, E. (1999). BOA: The Bayesian optimization algorithm. In Banzhaf, W., Daida, J., Eiben, A. E., Garzon, M. H., Honavar, V., Jakiela, M., and Smith, R. E., editors, *Proceedings of the Genetic and Evolutionary Computation Conference GECCO-1999*, volume I, pages 525–532, Orlando, FL. Morgan Kaufmann Publishers, San Francisco, CA.
- Pelikan, M. and Mühlenbein, H. (1999). The bivariate marginal distribution algorithm. In Roy, R., Furuhashi, T., and Chawdhry, P., editors, *Advances in Soft Computing - Engineering Design and Manufacturing*, pages 521–535, London. Springer.
- Pelikan, M., Sastry, K., and Cantú-Paz, E., editors (2006). *Scalable Optimization via Probabilistic Modeling: From Algorithms to Applications*. Studies in Computational Intelligence. Springer.
- Santana, R. (2003). A Markov network based factorized distribution algorithm for optimization. In *Proceedings of the 14th European Conference on Machine Learning (ECML-PKDD 2003)*, volume 2837 of *Lecture Notes in Artificial Intelligence*, pages 337–348, Dubrovnik, Croatia. Springer.
- Santana, R. (2005). Estimation of distribution algorithms with Kikuchi approximations. *Evolutionary Computation*, 13(1):67–97.
- Santana, R., Larrañaga, P., and Lozano, J. A. (2004). Protein folding in 2-dimensional lattices with estimation of distribution algorithms. In *Proceedings of the First International Symposium on Biological and Medical Data Analysis*, volume 3337 of *Lecture Notes in Computer Science*, pages 388–398, Barcelona. Springer.
- Santana, R., Larrañaga, P., and Lozano, J. A. (2005). Interactions and dependencies in estimation of distribution algorithms. In *Proceedings of the 2005 Congress on Evolutionary Computation CEC-2005*, pages 1418–1425, Edinburgh, U.K. IEEE Press.
- Santana, R., Larrañaga, P., and Lozano, J. A. (2007). The role of a priori information in the minimization of contact potentials by means of estimation of distribution algorithms. In Marchiori, E., Moore, J. H., and Rajapakse, J. C., editors, *Proceedings of the Fifth European Conference on Evolutionary Computation, Machine Learning and Data Mining in Bioinformatics*, volume 4447 of *Lecture Notes in Computer Science*, pages 247–257, Valencia, Spain. Springer.
- Santana, R., Larrañaga, P., and Lozano, J. A. (2008a). Estimation of distribution algorithms with affinity propagation methods. Technical Report EHU-KZAA-IK-1/08, Department of Computer Science and Artificial Intelligence, University of the Basque Country.
- Santana, R., Larrañaga, P., and Lozano, J. A. (2008b). Protein folding in simplified models with estimation of distribution algorithms. *IEEE Transactions on Evolutionary Computation*, 12(4):418–438.

- Santana, R., Larrañaga, P., and Lozano, J. A. (2009). Research topics on discrete estimation of distribution algorithms. *Memetic Computing*, 1(1):35–54.
- Santana, R., Ochoa, A., and Soto, M. R. (2001). The mixture of trees factorized distribution algorithm. In *Proceedings of the Genetic and Evolutionary Computation Conference GECCO-2001*, pages 543–550, San Francisco, CA. Morgan Kaufmann Publishers.
- Schwarz, G. (1978). Estimating the dimension of a model. *Annals of Statistics*, 7(2):461–464.
- Selman, B., Levesque, H., and Mitchell, D. (1992). A new method for solving hard satisfiability problems. In American Association for Artificial Intelligence, C., editor, *Proceedings of the 10th National Conference on Artificial Intelligence (AAAI-92)*, pages 440–446, San Jose, CA, USA. American Association for Artificial Intelligence. Menlo Park, CA, USA.
- Shakya, S. and McCall, J. (2007). Optimization by estimation of distribution with DEUM framework based on Markov random fields. *International Journal of Automation and Computing*, 4(3):262–272.
- Shakya, S. and Santana, R. (2008). An EDA based on local Markov property and Gibbs sampling. In Keijzer, M., editor, *Proceedings of the 2008 Genetic and Evolutionary Computation Conference (GECCO)*, pages 475–476, New York, NY, USA. ACM.